



Abusing Go Syntax to Create a Domain-Specific Language

Go is often the tool of choice for building the guts of a high-performance system, but Go was also designed with some features that are great for building high-level abstractions. You don't need to switch gears to a dynamic language like Ruby or Python to enjoy pleasant APIs or declarative syntax.

It's an increasingly popular choice to express an API as a DSL – a Domain-Specific Language. A DSL is a language-within-a-language that is compiled or interpreted inside a host language– in our case, Go. Through clever API design, A DSL begins to look like its own language specially suited to a particular task. Some DSLs like CSS and SQL are built as stand-alone languages with their own parsers, but for now we'll focus on the ones we can build for the Go compiler, to use within Go code.

DSLs are used for infrastructure automation, data model declaration, query building, and tons more applications. It can be pleasant to write in a DSL for these types of connecting-and-configuring tasks because they offer a *declarative* syntax. Rather than *imperatively* describing all of the logic and operations needed to get your application into a particular state, a DSL lets you declare the desired structure and attributes of that state, and its underlying implementation takes the steps to get there. The resulting code tends to be easier to read, too.

We're going to look at how to construct APIs that are valid Go that accepted by the Go compiler, but that begin to feel like their own language. Now, this article is called “Abusing Go Syntax to Create a DSL” because, well, we might violate the spirit of Go just a bit in the interest of bending the letter of its laws to our will. I hope you'll be judicious in applying the more questionable practices discussed here. I also hope that by exploring them, you'll be inspired to think creatively about how to write expressive Go APIs that are fun to work with and easy to understand.

A Motivating Example

We're going to build a simple DSL for constructing HTTP middleware. This domain is a great candidate for a DSL because it's full of common, well-understood and often reused patterns like access restrictions, rate limiting, session handling, and more. It would be better for both readers and writers of the code that implements those patterns if that code read more like a declarative config file than like an imperative reinvention of the wheel.

Type Identities

One subtly powerful feature of Go for writing code with a descriptive feel is its [type identities](#). Most of the time when declaring our own types in Go, we're declaring a struct or interface. We can also declare new type identities for existing types, referring to them by a new name we choose.

This can be something simple like giving a new name to a basic type, such as a `string` type for host names:

```
1 type Host string
```

We can also create type identities for collections:

```
1 type HostList []Host
2 type HostSet map[Host]interface{}
```

Now anywhere in our code, a variable of type `HostList` will really be a `[]Host`, or really a `[]string` under the hood, but with a more descriptive name.

A benefit of these type identities, apart from cosmetics and saved keystrokes, is that these new types can be enriched with their own methods. For example:

```
1 func (s HostSet) Add(n Host) {
2     s[n] = struct{}{}
3 }
4
5 func (s HostSet) Remove(n Host) {
6     delete(s, n)
7 }
8
9 func (s HostSet) Contains(n Host) bool {
10    _, found := s[n]
11    return found
12 }
```

Now we can use a `HostSet` as though it were a more complex container struct accessed via methods:

```
1 func main() {
2     s := make(HostSet)
3     s.Add("golang.org")
4     s.Add("google.com")
5     s.Add("gopheracademy.org")
6     s.Remove("google.com")
7
8     hostnames := HostList{
9         "golang.org",
10        "google.com",
11        "gopheracademy.org",
12    }
13    for _, n := range hostnames {
14        fmt.Printf("%s? %v\n", n, s.Contains(n))
15    }
16 }
```

This gives output:

```
1  golang.org? true
2  google.com? false
3  gopheracademy.org? true
```

[Try it out here.](#)

What have we gained here? We've created an abstraction over a simple `map`; we can use it like a set – it has `Add` and `Remove` operations, and a `Contains` check – we've created idioms that can be reused throughout our code. It's better-encapsulated than passing around a `map[string]interface{}` and hoping the “set of hostnames” semantics are honored when accessing the map. It's also more fluent and descriptive than, say

```
1  func SetContains(s map[string]interface{}, hostname string) bool {
2      _, found := s[hostname]
3      return found
4  }
5
6  func main() {
7      s := make(map[string]interface{})
8      if SetContains(s, hostname) {
9          // do stuff
10     }
11 }
```

Experiment a bit with creating new type identities, particularly for different slice and map types, and even for channels. What idioms can you create to make working with these types simpler and clearer?

Higher-Order Functions

Go incorporates some concepts from functional programming that are invaluable for creating expressive, declarative APIs. Go offers the ability to assign functions to variables, to pass a function as an argument to another function, and to create anonymous functions and [closures](#). Using *higher-order functions* that create, modify, or compose the behavior of other functions, you can easily combine pieces of logic and functionality into a more sophisticated whole with just a few statements, rather than by duplicating code or creating a tangle of conditional logic.

Let's build on our example from above. Let's add a method to `HostList` that takes a function as an input and returns a new `HostList`:

```
1  func (l HostList) Select(f func(Host) bool) HostList {
2      result := make(HostList, 0, len(l))
3      for _, h := range l {
4          if f(h) {
5              result = append(result, h)
6          }
7      }
8      return result
9  }
```

This method of `HostList` has the effect of creating a new `HostList` for which the provided condition (func `f`) is `true`. Let's make a simple condition function to plug in for `f`:

```

1 // import "strings"
2 func IsDotOrg(h Host) bool {
3     return strings.HasSuffix(string(h), ".org")
4 }

```

and use it in our new method of `HostList`:

```

1 myHosts := HostList{"golang.org", "google.com", "gopheracademy.org"}
2 fmt.Printf("%v\n", myHosts.Select(IsDotOrg))

```

we see output:

```

1 [golang.org gopheracademy.org]

```

`Select` returned only those elements of `myHosts` for which the function we passed into it, `IsDotOrg`, was `true`, the hostnames that contained `.org`.

`func(Host) bool` is a bit gnarly as a parameter type and makes the method signature of `Select` difficult to read, so let's use our type identity trick to make it a bit neater:

```

1 type HostFilter func(Host) bool

```

This makes the signature of `Select` a bit more readable:

```

1 func (l HostList) Select(f HostFilter) HostList {
2     //...
3 }

```

and has the added benefit that we can declare some methods of `HostFilters`:

```

1 func (f HostFilter) Or(g HostFilter) HostFilter {
2     return func(h Host) bool {
3         return f(h) || g(h)
4     }
5 }
6
7 func (f HostFilter) And(g HostFilter) HostFilter {
8     return func(h Host) bool {
9         return f(h) && g(h)
10    }
11 }

```

If we want to declare a function that can use these `HostFilter` methods, unfortunately we will need to go a bit out of our way to do so. For a function to be a valid receiver of `HostFilter` methods, it's not sufficient to match the signature of a `HostFilter`, we need to declare the function as a `HostFilter` explicitly:

```

1 var IsDotOrg HostFilter = func(h Host) bool {
2     return strings.HasSuffix(string(h), ".org")
3 }

```

Unfortunately here it becomes clear that we have begun to make good on our threat to “abuse” Go's syntax. Declaring a function by assigning an anonymous function to a variable gives an unclean feeling. Note that this isn't required to use higher-order functions, or to take advantage of a type identity for a function signature – any `func(Host) bool` can be assigned to a `HostFilter` variable or parameter. This hare-brained declaration is only needed to be able to use functions like `IsDotOrg` as the receiver of the `HostFilter` methods.

The payoff, though, is that going to these lengths to allow our `HostFilter` functions to use these methods enables an interesting syntax:

```
1 var HasGo HostFilter = func (h Host) bool {
2     return strings.Contains(string(h), "go")
3 }
4
5 var IsAcademic HostFilter = func(h Host) bool {
6     return strings.Contains(string(h), "academy")
7 }
8
9 func main() {
10     myHosts := HostList{"golang.org", "google.com", "gopheracademy.org"}
11     goHosts := myHosts.Select(IsDotOrg.Or(HasGo))
12     academies := myHosts.Select(IsDotOrg.And(IsAcademic))
13
14     fmt.Printf("Go sites: %v\n", goHosts)
15     fmt.Printf("Academies: %v\n", academies)
16 }
```

Running this gets:

```
1 Go sites: [golang.org google.com gopheracademy.org]
2 Academies: [gopheracademy.org]
```

We can see a language of our own taking shape in an expression like `myHosts.Select(IsDotOrg.Or(HasGo))`. It reads a bit like English, if a little like something you might hear in the swamps of Dagobah. The declarative syntax has begun to emerge – the expression says more about the desired result (“select the elements of `myHosts` that are `.org`s or contain ‘Go’”) than it does about the specific steps required to get there. We used higher-order functions, `Select`, `And`, and `Or`, to compose behaviors from three different pieces of code in an entirely dynamic way.

This is a powerful way of expressing behavior, but all of this method chaining can start to get muddled:

```
1 // etc.
2 myHosts.Select(IsDotOrg.Or(HasGo).Or(IsAcademic).Or(WelcomesGophers).And(UsesSSL))
```

So perhaps we can clean things up by using a variadic method:

```
1 var HostFilter Or = func (clauses ...HostFilter) HostFilter {
2     var f HostFilter = nil
3     for _, c := range clauses {
4         f = f.Or(c)
5     }
6     return f
7 }
```

and then rewrite the chained invocation above as:

```
1 myHosts.Select(Or(IsDotOrg, HasGo, IsAcademic, WelcomesGophers).And(UsesSSL))
```

Another warning: these functional-style constructs are some of the most dangerously powerful features of Go – all of the truly unreadable Go I’ve ever read and most of the truly unreadable Go I’ve ever written got to be that way by abusing these features, creating anonymous functions and passing them through layer after layer of indirection.

Nevertheless, the dynamism of functional-style programming is invaluable when building our own language inside of Go. Higher-order functions, functions that operate on other functions and return whole new functions, afford us the ability to compose or parameterize behaviors. The purpose of creating a DSL is to simplify the solutions to a class of problems by exposing to the DSL's user a few concepts useful for solving those problems and empowering them configure and combine those concepts in meaningful ways. Composable dynamic behaviors created through higher-order programming are one way deliver that functionality.

A More Useful Example

We'll build on our work on hostnames to make something a little closer to what we might use in a real application. Importing the `net/http` package, let's create another type identity:

```
1 type RequestFilter func(*http.Request) bool
```

We can use a `RequestFilter` in a simple HTTP server to evaluate whether a given `http.Request` satisfies a particular condition, as we did with `HostFilter` above. We can use those conditions to determine whether to handle or reject the request.

We'll shift from working with hostnames as above to working with ranges of IP addresses. We'll use `CIDR` blocks, e.g. `"192.168.0.0/16"`, which identifies a range of IPs from `192.168.0.0` through `192.168.255.255`. We'll create a `RequestFilter` that filters requests based on IP.

From the `net` package, we'll use the `ParseCIDR` function to parse the CIDRs, and the `ParseIP` function to parse IP addresses from incoming requests. One of the return values from `ParseCIDR` is an `IPNet` which conveniently has a `Contains` method that will do the work of telling us whether the incoming IP matches the range in our CIDR block.

So let's also import the `net` package and write a `RequestFilter` that takes a variadic input of CIDR blocks in string form:

```
1 func CIDR(cidrs ...string) RequestFilter {
2     nets := make([]*net.IPNet, len(cidrs))
3     for i, cidr := range cidrs {
4         // TODO: handle err
5         _, nets[i], _ = net.ParseCIDR(cidr)
6     }
7     return func(r *http.Request) bool {
8         // TODO: handle err
9         host, _, _ := net.SplitHostPort(r.RemoteAddr)
10        ip := net.ParseIP(host)
11        for _, net := range nets {
12            if net.Contains(ip) {
13                return true
14            }
15        }
16        return false
17    }
18 }
```

Note that the `net/http` package already contains a type for HTTP handlers, `HandlerFunc`:

```
1 type HandlerFunc func(ResponseWriter, *Request)
```

and we'll be using higher-order functions and our `RequestFilters` to modify `http.HandlerFuncs`, so let's declare a type for functions that operate on `http.HandlerFuncs`:

```
1 type Middleware func(http.HandlerFunc) http.HandlerFunc
```

and let's make some functions to build `Middleware` that uses the `RequestFilter`:

```
1 func Allow(f RequestFilter) Middleware {
2     return func(h http.HandlerFunc) http.HandlerFunc {
3         return func(w http.ResponseWriter, r *http.Request) {
4             if f(r) {
5                 h(w, r)
6             } else {
7                 // TODO
8                 w.WriteHeader(http.StatusForbidden)
9             }
10        }
11    }
12 }
```

So now, for example, you could modify an HTTP handler `MyHandler` to only accept requests from `127.0.0.1` with something like:

```
1 filteredHandler := Allow(CIDR("127.0.0.1/32"))(MyHandler)
```

Let's try it by running a simple server:

```
1 func hello(w http.ResponseWriter, r *http.Request) {
2     fmt.Fprintf(w, "Hello\n")
3 }
4
5 func main() {
6     http.HandleFunc("/hello", Allow(CIDR("127.0.0.1/32"))(hello))
7     log.Fatal(http.ListenAndServe(":1217", nil))
8 }
```

If you hit your new endpoint from your local machine at <http://0.0.0.0:1217/hello>, you should see “Hello” in response; if you hit it from another IP address, you should see a `403 Forbidden` error.

For fun, let's add another kind of `RequestFilter` that implements a really naive authentication mechanism:

```
1 func PasswordHeader(password string) RequestFilter {
2     return func(r *http.Request) bool {
3         return r.Header.Get("X-Password") == password
4     }
5 }
```

and one based on HTTP method:

```

1 func Method(methods ...string) RequestFilter {
2     return func(r *http.Request) bool {
3         for _, m := range methods {
4             if r.Method == m {
5                 return true
6             }
7         }
8         return false
9     }
10 }

```

and a **Middleware** that performs some simple logging:

```

1 func Logging(f http.HandlerFunc) http.HandlerFunc {
2     return func(w http.ResponseWriter, r *http.Request) {
3         fmt.Printf("[%v] - %s %s\n", time.Now(), r.Method,
4             r.RequestURI)
5         f(w, r)
6     }
7 }

```

which we can try with an update to our server:

```

1 func main() {
2     http.HandleFunc("/hello", Logging(Allow(CIDR("127.0.0.1/32"))(hello)))
3     log.Fatal(http.ListenAndServe(":1217", nil))
4 }

```

Run this and visit <http://localhost:1217/hello> a few times in your browser and in the console where the server is running you should see:

```

1 [2016-12-14 07:42:12.022266374 -0500 EST] - GET /hello
2 [2016-12-14 07:42:14.537985456 -0500 EST] - GET /hello
3 [2016-12-14 07:42:24.220089221 -0500 EST] - GET /hello

```

This syntax is fairly declarative as is, but the method chaining can get a little awkward. Methods have to be chained in the right order to behave correctly, and the result can be difficult to read.

We can use a struct to further flesh out our DSL and give our users an even cleaner way to declare their middleware configuration:

```

1 type Filters []RequestFilters
2 type Stack []Middleware
3 type Endpoint struct {
4     Handler    http.HandlerFunc
5     Allow      Filters
6     Middleware Stack
7 }

```

then we could express the endpoint above with the same restrictions as:

```

1 var MyEndpoint = Endpoint{
2     Handler: hello,
3     Allow: Filters{
4         CIDR("127.0.0.1/32"),
5     },
6     Middleware: Stack{
7         Logging,
8     },
9 }

```


Which is much easier to write, read, and modify. We just need to add a few methods to our struct and type identities turn this declarative description into a usable `http.HandlerFunc`:

```
1 // Combine creates a RequestFilter that is the conjunction
2 // of all the RequestFilters in f.
3 func (f Filters) Combine() RequestFilter {
4     return func(r *http.Request) bool {
5         for _, filter := range f {
6             if !filter(r) {
7                 return false
8             }
9         }
10        return true
11    }
12 }
13
14 // Apply returns an http.HandlerFunc that has had all of the
15 // Middleware functions in s, if any, to f.
16 func (s Stack) Apply(f http.HandlerFunc) http.HandlerFunc {
17     g := f
18     for _, middleware := range s {
19         g = middleware(g)
20     }
21     return g
22 }
23
24 // Builds the endpoint described by e, by applying
25 // access restrictions and other middleware.
26 func (e Endpoint) Build() http.HandlerFunc {
27     allowFilter := e.Allow.Combine()
28     restricted := Allow(allowFilter)(e.Handler)
29
30     return e.Middleware.Apply(restricted)
31 }
```

and, finally, modify the server to use the endpoint built this way:

```
1 func main() {
2     http.HandleFunc("/hello", mw.MyEndpoint.Build())
3     log.Fatal(http.ListenAndServe(":1217", nil))
4 }
```

To see the benefit of this mini-DSL we've created, let's add one more kind of middleware:

```
1 func SetHeader(key, value string) Middleware {
2     return func(f http.HandlerFunc) http.HandlerFunc {
3         return func(w http.ResponseWriter, r *RequestFilter) {
4             w.Header().Set(key, value)
5             f(w, r)
6         }
7     }
8 }
```

And then add it, along with another `RequestFilter`, to our endpoint:

```

1  var MyEndpoint = Endpoint{
2      Handler: hello,
3      Allow: Filters{
4          CIDR("127.0.0.1/32"),
5          PasswordHeader("opensesame"), // added
6          Method("GET"), // added
7      },
8      Middleware: Stack{
9          Logging,
10         SetHeader("X-Foo", "Bar"), // added
11     },
12 }

```

We've added significantly to the complexity of `MyEndpoint` without adding much complexity to its declaration.

This is a useful DSL for building single HTTP endpoints, but frequently we'll want more than just one on a service. We'll add one last element to our demo DSL, a way to create several routes and their endpoints at once:

```

1  type Routes map[string]Endpoint
2
3  func (r Routes) Serve(addr string) error {
4      mux := http.NewServeMux()
5      for pattern, endpoint := range r {
6          mux.Handle(pattern, endpoint.Build())
7      }
8
9      return http.ListenAndServe(addr, mux)
10 }

```

and then our service becomes:

```

1  func main() {
2      routes := Routes{
3          "/hello": {
4              Handler: hello,
5              Middleware: Stack{
6                  Logging,
7              },
8          },
9          "/private": {
10             Handler: hello,
11             Allow: Filters{
12                 CIDR("127.0.0.1/32"),
13                 PasswordHeader("opensesame"),
14             },
15             Middleware: Stack{
16                 Logging,
17             },
18         },
19         "/test": {
20             Handler: hello,
21             Middleware: Stack{
22                 Logging,
23                 SetHeader("X-Foo", "Bar"),
24             },
25         },
26     }
27     log.Fatal(routes.Serve(":1217"))
28 }

```

Note that Go automatically infers the type of the `Endpoint` struct literals in the `Routes` map, saving us even more typing and clutter.

This HTTP middleware DSL shows how much can be accomplished in a relatively small amount of Go, but it's a toy example. Here are some ideas for exercises to extend it and to make the DSL even more powerful:

- Implement additional `RequestFilters`, like a rate-limiter, perhaps using golang.org/x/time/rate or juju/ratelimit, or a more robust authentication mechanism
- Implement another `Middleware`
- Modify the `Endpoint` struct to include a `Deny` field of type `Filters`, that rejects the request if any of its `RequestFilters` is `true`
- Each of the endpoints in the final sample included `Logging` in its middleware; add to the DSL a facility to apply a set of common restrictions or middleware to all of the endpoints.
- Create a way for this middleware stack to create a `context.Context` and to work with handlers that accept them.

To recap, we used type identities to create abstractions over simple collection types and functions of particular signatures, and we took advantage of Go syntax features like variadic functions and inferred types to write a smooth, uncluttered syntax. The heavy lifting in creating our DSL was performed by higher-order functions that let us create parameterized behaviors that could be combined and configured at runtime. We employed a few dangerous coding practices to do it, but as long as we apply them only when reducing complexity for end-users is the right tradeoff, we can all sleep at night.

The Go you get out of the box is detail-oriented, minimalistic, and can become verbose. Go gives you the tools, however, to build up your own abstractions- your own high-level language- to write code that is as pithy, elegant, and expressive as any you'll find in a dynamic or purely functional language, but that still gives us access to all of the features we love about Go.

[« Advanced Encoding and Decoding Techniques in Go »](#)

[Writing good unit tests for SOLID Go »](#)

Explore →

[bleve](#) [blog](#) [community](#) [deployment](#) [git](#) [go](#) [gopher-academy](#) [gophercon](#) [meta](#)
[releases](#) [revel](#) [search](#) [text](#)